BEYOND 2030



IEEE Lumen 2.0 specification book

IEEE CS ISTIC SB X IEEE CS SESAME SB

WELCOME TO LUMEN 2.0

Hey there! You're holding the official specification book for the Beyond 2030 challenge. Everything you need to know — from what we expect to how you'll be scored — is right here.

This book has six parts, each answering a different question you might have:

Part I — The Challenge: what Beyond 2030 is, and why it's wide open for you to explore.

Part II — The Build: the philosophy behind problem selection, tech choices, and AI.

Part III — The Rules: every requirement you must follow, all in one place.

Part IV — Evaluation: exactly how your project will be scored (100 points total).

Part V — The Pitch Session: how you'll pitch and defend your project.

Part VI — Closing: what sets winning projects apart, and the spirit of the challenge.

Quick tip: Read Parts I–III before you start building — Part III especially, since everything there is mandatory. Check Part IV while you build to make sure you're on track. Read Part V before you pitch.

PART I — THE CHALLENGE

1 What Is Beyond 2030?

As civilization reaches toward the Moon, Mars, orbital habitats, and eventually deeper into space, entirely new challenges will emerge — challenges no one has solved yet.

Beyond 2030 is your chance to find one of those future problems and build a real technological solution for it.

We're not handing you a specific problem to solve. You get to decide everything:

What problem to tackle

Who it affects

Why it matters

How it might emerge in the future

What the solution should do

Which technologies to use

How to design and build it

The goal isn't just to build software. The goal is to imagine a future, spot a real problem inside it, and create something that solves it.

2 The Mission — Humanity Beyond Earth

The big theme is humanity's expansion beyond Earth. Yourteam's job is to identify a problem that could emerge as humans build communities, systems, andinfrastructure beyond our planet.

Here are some areas you could explore (but feel free to gobeyond this list):

Lunar and Mars settlements, orbital habitats, and space stations

Interplanetary transportation, logistics, and Earth–space supply chains

Space manufacturing, agriculture, and resource extraction

Human–robot collaboration and autonomous systems

Space communication and cybersecurity

Astronaut health, wellbeing, and emergency response

Energy, water, and waste management

Governance, education, and workforce management

Scientific research and space debris management

Earth–Moon–Mars coordination and planetary infrastructure

These are just starting points — the most creative solutions often come from problems nobody expected.

3 How Far Into the Future?

The name Beyond 2030 marks the start of a future era, not a deadline. Your problem could emerge in 2030, 2050, 2100, or even further out — the further you project, the stronger your reasoning needs to be.

Your submission should walk through this thinking:

- What future scenario does your team see happening?
- What developments lead to that scenario?
- What new problem shows up as a result?
- Who is affected?
- Why won't today's solutions work?
- Why could your solution handle it?

We reward well-reasoned possibilities, not wild guesses. You don't have to be right — you have to be convincing.

4 Freedom Is the Challenge

We deliberately didn't hand you a specific problem. You're not given something to solve — you have to discover it yourself. That freedom extends to everything:

- No predefined app that every team must copy
- No mandatory tech stack or programming language
- No requirement to use AI
- No single "right" answer

That freedom is part of the challenge itself. You're being evaluated on your ability to recognize an opportunity, frame a real problem, make smart decisions, and turn an original idea into something real.

PART II — THE BUILD

5 What We Expect

Your team needs to build aworking software solution for the problem you've identified. Here's what it should do:

- Solve a clearly defined problem with real users or stakeholders

Actually work — not just look good

Show a clear connection between the problem and your solution

Have a core feature that works end to end in the live demo

Come with a clear explanation of how it's built

Your solution should lean toward a SaaS-style product philosophy — a usable product or service built for defined users, not a one-off script — but we're not restricting your architecture. It could be a web platform, cloud service, mobile app, desktop app, IoT system, simulation, or anything else — as long as it tackles the challenge in a meaningful way.

6 Technology Is a Tool, Not a Trophy

Use whatever technology helps your solution — AI/ML, IoT, cybersecurity, robotics, cloud, data engineering, computer vision, simulation, AR/VR, blockchain, distributed systems, digital twins, embedded systems, or anything else that makes sense.

Technology should serve your problem — not the other way around.

Adding a technology just to make your project look more advanced doesn't earnextra points. A simple solution that nails a hard problem can easily beat a complex one that doesn't actually solve anything.

For every major technology in your stack, ask yourself:

"Why does this belong in our solution?"

7 The "2030+" Test

Every project should pass three simple questions:

Why will this problem exist? What future development creates it?

Why does it matter? Who gets hurt, and what happens if nobody solves it?

Why is your solution the right one? Why does it actually address the problem?

You don't need to prove your prediction will come true. You need to show that your scenario is coherent, well-reasoned, and technically grounded.

8 Using AI

AI is allowed. Understanding is mandatory.

Go ahead and use AI however you want — coding assistants, generative AI, APIs, ML models, AI dev tools, AI-generated assets, AI research. There's zero requirement to use AI in your project.

But if you use AI to help build it, you still need to understand what you built. That's a rule, not a suggestion. See Rule 12.5 for the details.

Being able to generate code is not the same as understanding the system that code creates.

9 Originality

Of course you canuse existing tools as building blocks — open-source libraries, frameworks, APIs, datasets, AI models, dev tools, and other public resources. Using existing technology is expected. Building nothing of your own is not acceptable.

Here's the question you should be able to answer:

"What did we create that wasn't already there?"

See Rule 12.4 for how to properly credit borrowed work.

10 Technical Freedom

There is no mandatory tech stack.Pick the tools that make the most sense for your project. Your choices should be justified by whatthe project actually needs — not by what sounds impressive.

Teams choosing simple, well-justified technologies are absolutely not at a disadvantage.

11 Our Development Philosophy

This challenge isn't about who can write the most code. Modern tools — especially AI — have made building sophisticated software more accessible than ever. And that's the point: as implementation gets easier, your ability to find meaningful problems and imagine valuable solutions matters even more.

We put serious weight on creativity, problem selection, future thinking, product design, user experience, and your understanding of what you built.

A technically impressive product solving a tiny problem shouldn't automatically outscore a simple, brilliantly original solution tackling a major future challenge.

PART III — THE RULES

12 Rules Overview

Every binding rule is here. Everything else in this book is philosophy, guidance, and context. If another section sounds like a requirement, the rule itself is spelled out in this part.

Seven groups of rules follow: eligibility, submission, functionality, attribution, AI transparency, conduct, and the pitch session.

12.1 Eligibility

The challenge is open to everyone — no restrictions on university, field of study, or technical background.

- Your team must have 2 to 5 members.
- You cannot participate solo.
- Teammates from different schools or institutions? Totally fine.

We encourage interdisciplinary teams — the strongest solutions don't always come from the strongest coders.

12.2 What to Submit

Required:

- A working software solution and its source code
- Project description, problem definition, and who the target users are
- Your future scenario and why it's relevant
- Description of your solution and tech stack
- A live demonstration of the working product

Recommended (not mandatory, but they help):

- Architecture diagram and technical docs
- Supporting research, references, or evidence
- Impact analysis
- Business or adoption model

Extra material won't save a product that doesn't work.

12.3 It Must Work

Your solution must be demonstrable. The core feature must work end to end during the live demo. It doesn't need to be production-ready — but it does need to prove that your central idea has actually been implemented.

A prototype is acceptable. A mockup pretending to be a working product is not.

12.4 Originality & Attribution

You must clearly credit any significantexternal work in your project — open-source libraries, frameworks, APIs, datasets, AI models,dev tools. Cloning an existing product without adding real value of your own is not acceptable.

Being transparent about what you built vs. what you borrowed is required — not optional.

12.5 AI Transparency

There is no requirement touse AI. But if you do, your team is fully responsible for the result and must understand it well enough to explain it.

Judges may ask you to walk through:

- Your architecture and algorithms
- AI models, prompts, or workflows you used
- APIs and dependencies
- Implementation choices and security decisions
- Data processing pipeline

These questions apply to AI-built parts just as much as anything else in your project.

12.6 Conduct & Compliance

You're responsible for makingsureyour project:

- Respects applicable lawsandregulations
- Doesn't intentionally harmusers
- Doesn't infringe intellectualproperty rights
- Contains no malicious functionality
- Properly credits significant external resources (see Rule 12.4)
- Can be honestly presented to the jury

Don’t knowingly misrepresent what your project can do, how original it is, or how it’s built.

12.7 Pitch Session

Projects are presented atthe pitch session on 25 October at 9:00 AM. Each team gets:

- 5 minutes to pitch (your demo is part of this time)
- 5 minutes of Q&A with the jury

See Sections 15 and 16 for what to cover.

PART IV — EVALUATION

13 How You’ll Be Scored

Your projectwill be evaluated acrossfive dimensions. The total is 100 points:

Criterion	Points
Innovation & Creativity	25
SaaS / Solution Quality	30
Future Relevance	20
Problem Significance	10
Technical Understanding & Defense	15
TOTAL	100

13.1 Innovation & Creativity — 25 pts

Judges look at: how original your concept is, how creative your problem is, the novelty of your approach, how well you interpreted the challenge, creative use of technology, how your solution stands apart from what already exists, and whether it could change how people think about the problem.

The most innovative solution isn't always the most technically complex one.

13.2 SaaS / Solution Quality — 30 pts

This is the biggest category — it's all about thequality of what you actually built:

- Functionality — does it work?
- Effectiveness — does it really solve the problem you defined?
- Technical implementation — is it coherent and well-chosen?
- UX/UI — is it intuitive, usable, and thoughtfully designed?
- Reliability — does it hold up during the live demo?
- Product coherence — do the features all serve the mission?
- Execution — how well did you turn your idea into reality?

A pretty interface can't save a product that doesn't solve its problem. And impressive engineering can't save something that's impossible to use.

13.3 Future Relevance — 20 pts

Judges look at: how plausible your futurescenario is, how it connects to humanity's expansion beyond Earth, your understanding of future tech or societal shifts, the strength of your reasoning, and how well you explain why this problem will matter.

You don't need to be a fortune teller — you need to build a convincing case for the future you've chosen.

13.4 Problem Significance — 10 pts

Judges look at: how severe the problem is, how many or how important the affected people are, what the consequences of leaving it unsolved would be, whether current solutions exist, and how relevant it is to life beyond Earth.

A problem doesn't have to affect millions to be significant. A specialized problem affecting astronauts, researchers, or critical infrastructure can carry enormous weight.

13.5 Technical Understanding & Defense — 15 pts

You need to prove you understand what you built. Judges can askabout architecture, code, algorithms, databases, APIs, AI systems, security, infrastructure, data, tech choices, limitations, scalability, and development decisions.

Be ready to explain WHY your system works — not just show THAT it works.AsAI makes building easier, the ability to defend what you built matters more thanever.

14 Judging Principle

The scoring framework is a guide, not a rigid formula. Judges use their professional judgment and look at the full picture:

Problem → Future → Idea → Technology → Product → Impact

Your final score reflects the overall strength and coherence of your project — not just individual checkboxes.

PART V — THE PITCH SESSION

15 The Pitch

As a finalist, you'llpresent your project to the jury at the pitch session on 25 October at 9:00 AM. You have 5 minutes,and your demo counts toward them. Think of it as telling a story — not just describing features.Your pitch should cover:

- The Future — what future scenario does your team see?
- The Problem — what goes wrong in that future?
- The Users — who suffers from this problem?
- The Solution — what did you build to fix it?
- The Technology — how does it work under the hood?
- The Impact — what changes if your solution succeeds?
- The Demo — show us the product in action.

16 Technical Defense

After your 5-minute pitch, you get 5 minutes of Q&A with the jury. They may dig into any part of your project: architecture, implementation, tech choices, AI usage, security, data handling, scalability, product decisions, assumptions about the future, known limitations, or your development process.

Your project doesn't have to be perfect. But you should know exactly where it's not — and be able to explain why.

PART VI — CLOSING

17 What Wins, and What Doesn't

There's no magic formula. A winning project might be technically sophisticated or remarkably simple. AI-powered or completely AI-free. IoT-heavy or built on a single clever algorithm. Focused on UX, infrastructure, or a human problem nobody else noticed.

What makes a great project? When the jury thinks: "I never thought about this problem that way" — and then: "and they actually built something that could solve it."

Yourproject won't score higher just because it:

- Uses AI
- Uses many technologies
- Has a massive codebase
- Has complicated architecture
- Uses expensive cloud infrastructure
- Looks futuristic

Complexity is not innovation. Code volume is not impact. Technology is not the idea.

18 The Spirit of Beyond 2030

The future doesn't need more technologyfor the sake of technology. It needs people who know what technology should be built.

AI can now help you write code, generate interfaces, build prototypes, and accelerate development. But no tool will ever automatically answer the questions that matter most:

What should we build? Why? Who needs it? What future does it create?

Beyond 2030 puts those questions at the center. You're not competing to build the most complex app — you're competing to show that you can imagine, question, identify, design, build, and defend. The challenge starts before the first line of code.

Humanity has always expanded by solving problems it had never faced before. The next frontier will be no different. As we leave Earth, new environments will create new constraints, new systems, new opportunities, and entirely new problems — some of which seem impossible today, and some of which don't even exist yet.

Your mission: find one. Then build the solution.

THE FUTURE IS YOURS TO BUILD.

May the code be with you.